

Agentic AI Incident Response Platform for Cloud Applications

A plain-English walkthrough of what the project is, why it matters, how it works, and how to explain it — written for a project guide, a reviewer, an interviewer, and a curious beginner all at once.

Domain: Artificial Intelligence

Sub-domain: Agentic AI • RAG • LLM Apps

Area: AIOps / Incident Response / DevOps Automation

01 Project Overview

In plain words: this project is an **AI teammate for the on-call engineer**. When something breaks in a cloud application — a service crashes, errors spike, checkout stops working — this platform notices it automatically, reads the logs and past history like a human would, figures out a likely cause, and suggests a fix. It never acts on its own for anything risky; a human always clicks "approve" before anything real happens. Every single thing the AI looked at and decided is written down, so nothing is a black box.

Underneath, it's a full backend platform (built in Java/Spring Boot) that any company's applications can plug into with a small SDK, plus an AI layer that investigates problems the way a senior engineer would: check the logs, check what changed recently, check the documentation, check if this happened before.

02 Problem Statement

Modern cloud applications produce a huge amount of data all the time — logs, error traces, performance metrics, deployment records, alerts. When something goes wrong, an engineer has to manually:

- Open a logging tool and search for errors around the time things broke
- Check a separate dashboard for metrics (CPU, memory, response time)
- Check the deployment history to see if a recent release caused it
- Search old tickets or documentation for "has this happened before?"
- Guess at a root cause, then decide on a fix — often under pressure, at 2 AM

This is slow (minutes to hours of pure investigation before any fix starts), inconsistent (depends on who's on call and how experienced they are), and error-prone (tired humans miss things, or apply the wrong fix and make it worse).

03 Proposed Solution

The platform automates the investigation part of incident response while keeping a human in charge of anything risky. It works in four layers:

1. Always-on ingestion

A lightweight SDK inside the application constantly ships logs, errors, and deployment events to the platform — so the data is already there the moment something breaks.

2. Automatic detection

The platform watches this stream and opens an "incident" the moment something looks wrong — e.g. a sudden spike in errors right after a deployment.

3. AI investigation (agent + RAG)

An AI agent reads the relevant logs, searches company runbooks and past incidents for similar problems (this searching is called RAG — explained in §9), and writes a root-cause hypothesis with a suggested fix.

4. Approval-gated automation

Safe actions (like sending a Slack alert) can happen automatically. Anything risky (like rolling back a deployment) is only a *suggestion* until a human clicks approve — and every decision, human or AI, is permanently logged.

The result: instead of an engineer starting from zero, they start from an AI-written case file — "here's probably what broke, here's why I think so, here's what I recommend" — and they just have to say yes or no.

04 Simple Real-Time Example: Payment Service Failure

Scenario: an e-commerce app deploys a new version of its payment service, and it starts throwing errors.

- 1 **Error happens.** The payment service starts throwing `NullPointerException` on checkout right after a deploy.

- 2 **SDK sends logs.** The Java SDK already running inside the payment service ships these error logs to the platform in real time — no manual step needed.

- 3 **Incident is created.** The platform notices the error rate jumped right after a new deployment and automatically opens an incident: "Payment service throwing NPEs after deploy v1.4.2."

- 4 **AI checks logs, deployment history, and runbooks.** The AI agent pulls the exact error stack trace, sees the deployment that happened 2 minutes earlier, and searches the company's runbooks and past incidents for anything similar.

- 5 **AI finds a probable root cause.** It finds a runbook titled "Null pointer errors after config changes" and notices this deploy changed a config flag — a strong match.

- 6 **AI suggests a rollback.** It proposes: "Roll back to v1.4.1" with its reasoning attached, and flags it as a high-risk action.

- 7 **Human approves.** The on-call engineer reads the AI's reasoning on the dashboard, agrees, and clicks Approve.

- 8 **System triggers automation.** The platform executes the approved rollback action (in this project, a safe simulated rollback — real infrastructure control is a future-scope item).

- 9 **Audit log and postmortem are created.** Every step — the error, the AI's reasoning, the approval, the action, the result — is saved permanently, and a draft postmortem summary is generated automatically.

05 Another Real-World Example: High Latency in a Food Delivery App

Scenario: a food delivery app's order service becomes slow during a lunchtime rush — nothing has crashed, but customers are waiting too long.

- **Detection:** the platform is also watching performance metrics, not just hard errors. It notices average response time for the order service climbing steadily and crossing a threshold.
- **Incident opens automatically:** "Order service latency degraded — response time 3x normal during peak load," tagged as MEDIUM severity (it's a slowdown, not a full outage).
- **AI investigates:** it checks recent traffic volume, recent deployments (none in this case), and resource metrics, and notices the order service's database connection pool usage is maxed out.
- **AI's hypothesis:** "Order service is likely bottlenecked on database connections during high traffic — no recent code change is the cause."
- **AI's suggestion:** a MEDIUM-risk action like "increase database connection pool size" or "scale up order-service replicas," each with reasoning attached.
- **Engineer response:** instead of spending 20 minutes cross-referencing three dashboards, the engineer sees the AI's finding immediately, confirms it makes sense, and approves the scale-up.

This example matters because it shows the platform isn't just "watching for crashes" — it understands degraded performance too, which is often harder for humans to notice quickly.

06 Who Will Use This Project?

BACKEND ENGINEERS

Get logs, errors, and deployments automatically tracked without building any of this themselves.

DEVOPS ENGINEERS

Use the automation and approval workflow to safely respond to infrastructure issues.

SRE TEAMS

Rely on the AI's investigation to cut down time-to-diagnosis during on-call incidents.

CLOUD OPERATIONS TEAMS

Get a single dashboard across every service instead of many disconnected tools.

STARTUPS

Get enterprise-style incident response without hiring a dedicated SRE team first.

07 Why This Project Is Useful

- **Faster diagnosis** — the AI reads logs and history in seconds, not the 20-40 minutes a human spends context-switching between tools.
 - **Less manual log-checking** — engineers start from a summary, not a raw wall of text.
 - **AI-assisted root cause analysis** — the platform proposes a "why," not just a "what."
 - **Automation with human safety** — nothing risky happens without a person approving it, so speed doesn't come at the cost of control.
 - **Better documentation** — every incident automatically produces a written record and draft postmortem, which teams often skip today because it's tedious.
 - **Directly relevant to product companies** — this is exactly the kind of AIOps / platform engineering problem that backend, DevOps, and SRE teams at real companies work on daily.
-

08 Core Features

Login & Org/Project System

Multi-tenant: each company (organization) has its own projects, users, and data, fully isolated from every other company.

Java SDK / Spring Boot Starter

A drop-in dependency any Java app adds to automatically start shipping logs, errors, and deployment events.

Log/Error/Event Ingestion

A REST API that receives and stores everything the SDK sends, scoped to the right project.

Incident Detection & Grouping

Rules that turn "a spike of similar errors" into one tracked Incident instead of hundreds of separate alerts.

AI Root Cause Analysis

The AI agent's write-up of what most likely caused the incident, with its reasoning shown, not hidden.

RAG over Runbooks & Past Incidents

The AI searches company documentation and history for anything relevant before answering — see §9.

Automation Workflow

Defines possible fix-actions (notify, create ticket, rollback) and their risk level.

Human Approval

Anything above LOW risk needs an explicit human click before it executes.

Audit Logs

An unchangeable record of every action — who (or which AI) did what, and why.

Dashboard

The single screen where a human sees incidents, the AI's investigation trail, and pending approvals.

09 AI Concepts Used

Each concept explained the moment it's introduced — no assumed background needed.

LLM (Large Language Model) — the AI model itself (like GPT or Claude). It reads text and writes text back — in this project, it reads logs/context and writes a diagnosis.

Agentic AI — instead of just answering one question, the AI is given a goal ("investigate this incident") and the ability to take multiple steps and use tools on its own to get there, deciding what to check next based on what it already found.

RAG (Retrieval-Augmented Generation) — before answering, the AI first *retrieves* relevant documents (runbooks, past incidents) and includes them in what it reads, so its answer is grounded in real company knowledge instead of guessing from general training.

Embeddings — a way of turning text into a list of numbers that captures its meaning, so a computer can compare "how similar" two pieces of text are, even if they use different words.

Vector Database — a database built to store embeddings and quickly find the ones that are most similar to a new piece of text. This is what makes RAG's "search by meaning" fast.

Semantic Search — searching by meaning instead of exact keyword matching. Searching "payment crash" can still find a runbook titled "checkout failure" because they mean similar things.

Tool Calling — the AI isn't limited to just talking; it can call real functions, like `search_logs()` or `search_runbooks()`, read the result, and decide what to do next based on it.

Multi-Agent Workflow — splitting the work across more than one specialized AI agent (e.g. one that investigates, one that writes the postmortem) instead of one agent trying to do everything.

Guardrails — limits placed on the AI so it can't do unlimited or unsafe things — e.g. a maximum number of steps, a list of allowed tools, and risk classification on every action it proposes.

Human-in-the-Loop — the core safety principle of this project: the AI can investigate and suggest freely, but a human must approve anything that could cause real damage before it happens.

10 System Architecture

The platform is built as two main pieces: a **Core Platform** (handles logins, projects, incoming data, and incidents) and an **AI Agent Service** (handles the investigation). They talk to each other through events, so the AI's work never blocks or slows down the main platform.

```
graph TD
    A[Spring Boot App (customer's service)] -- uses --> B[Java SDK / Spring Boot Starter]
    B -- "sends logs, errors, deployments" --> C[Ingestion API (Core Platform)]
    C --> D[(Database + Message Queue (PostgreSQL + RabbitMQ))]
    D --> E[Incident Engine (detects & creates incidents)]
    E --> F[AI Agent + RAG (investigates & proposes a fix)]
    F --> G[Automation + Human Approval (executes only after a "yes")]
    G --> H[Dashboard + Audit Logs (humans see and review everything)]
```

A cleaner, color-coded version of this diagram is in the [Appendix](#) (Diagram 1), along with ready-to-paste Mermaid code.

11 Detailed Workflow Diagram

This is the life of a single incident, start to finish — the same nine steps walked through in the payment-service example in §4, shown as a pipeline:

Error → Ingest → Detect → Investigate → Suggest → Approve → Execute → Audit → Postmortem

- **Error** — something goes wrong in the real application.
- **Ingest** — the SDK ships the evidence to the platform.
- **Detect** — the platform recognizes a pattern and opens an incident.
- **Investigate** — the AI agent reads logs, deployments, and runbooks.
- **Suggest** — the AI proposes a root cause and an action, with reasoning.
- **Approve** — a human reviews and decides yes or no.
- **Execute** — the approved action runs.
- **Audit** — every step of the above is permanently recorded.
- **Postmortem** — a written summary of what happened is generated for the team.

A polished version of this flow is [Appendix Diagram 2](#).

12 Tech Stack By Category

CATEGORY	TECHNOLOGIES
Backend	Java 21, Spring Boot 3, Spring Security, Spring Data JPA
Database	PostgreSQL, Flyway (schema migrations)
Event Processing	RabbitMQ (or Kafka)
AI	OpenAI API / Claude API, LangGraph or OpenAI Agents SDK, embeddings, vector search
Vector Database	pgvector (or Qdrant)
Frontend	React, TypeScript, Tailwind CSS, TanStack Query, Recharts
DevOps	Docker, Docker Compose, GitHub Actions
Cloud	AWS ECS/Fargate or Azure Container Apps, S3/Azure Blob, CloudWatch/Azure Monitor
Observability	OpenTelemetry, Prometheus, Grafana, structured logging
Security	JWT, API keys, RBAC, audit logs, human approval system

13 Module-Wise Explanation

Auth Module

Handles sign-up/login and issues JWT tokens so dashboard users can securely access their organization's data.

Organization/Project Module

Keeps every company's data separate. A project usually maps to one real application or service.

API Key Module

Issues secret keys that the SDK uses to authenticate, so ingestion doesn't need a human login.

Ingestion Module

Receives and stores logs, errors, and deployment events sent by the SDK.

Incident Module

Owns the lifecycle of an incident: open, investigating, awaiting approval, resolved.

AI Agent Module

Runs the investigation: gathers context, retrieves knowledge, reasons, and proposes actions.

Knowledge Base Module

Stores runbooks and past-incident summaries, chunked and embedded for the AI to search.

Automation Module

Defines possible actions, their risk level, and executes them once approved.

Audit Log Module

An append-only record of everything that happened — the platform's memory and evidence trail.

Dashboard Module

The React frontend where humans see incidents, the AI's reasoning, and approve or reject actions.

14 What Makes This Different From A Normal Web App?

A typical student "CRUD" project (create-read-update-delete forms over a database) doesn't need to solve any of these:

- **Real-time event ingestion** — handling a continuous stream of data from external applications, not just form submissions.
 - **AI reasoning** — the system has to make a judgment call, not just fetch and display rows.
 - **RAG** — combining a live database search with an AI model's answer, correctly and efficiently.
 - **Tool calling** — letting an AI safely and predictably trigger real functions in the codebase.
 - **Automation with guardrails** — building a system that can act, but proving it won't act unsafely.
 - **Cloud-shaped deployment** — designed from day one to run as separate services communicating over events, the way real production systems do.
 - **System design** — multi-tenancy, security boundaries (API keys vs. JWT), and an audit trail are all deliberate architectural decisions, not afterthoughts.
 - **SDK integration** — shipping a reusable library that other developers install into their own projects, not just a single app.
-

15 Final Demo Story

A script for a live college presentation, roughly 5–7 minutes.

- 1 Open the dashboard — show the empty incidents list on a sample e-commerce app already wired with the SDK.

- 2 Trigger a real error in the sample app (e.g. run a script that ships a bad deploy).

- 3 Show the platform detect it live — a new incident appears on the dashboard within seconds.

- 4 Open the incident and show the AI's investigation trail: which logs it looked at, which runbook it matched, its reasoning.

- 5 Show the AI's suggested fix and its risk level.

- 6 Click Approve — show the automation execute.

- 7 Show the audit log entry and the generated postmortem summary for that incident.

- 8 Close with the line: "This system doesn't just alert — it investigates like an engineer would, and only acts once a human agrees."

16 Interview Explanation (60 Seconds)

Say this out loud in ~60 seconds

"I built an AI-powered incident response platform for cloud applications. Modern apps generate huge volumes of logs and errors, and engineers spend a lot of time manually digging through tools to find what went wrong. My platform automates that investigation: a Java SDK ships logs and errors from any application, the platform detects incidents automatically, and then an AI agent investigates — it reads the relevant logs, searches company runbooks and past incidents using RAG, and proposes a root cause and a fix. The important part is safety: the AI never executes anything risky on its own — every risky action needs explicit human approval, and every decision, by the AI or the human, is permanently audit-logged. It's built with Spring Boot, PostgreSQL, an event-driven architecture, and an LLM-based agent with tool-calling — so it's a full backend and AI systems project, not just a chatbot wrapper."

17 Future Scope

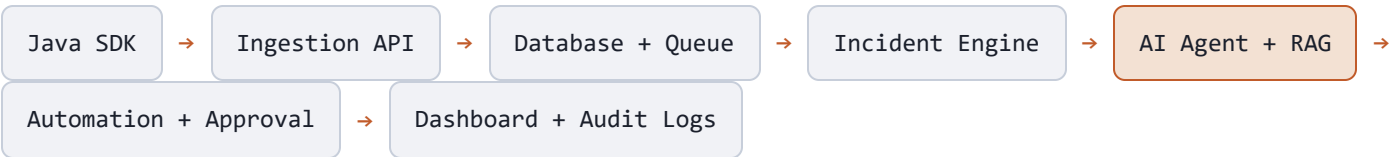
- **Kubernetes integration** — real rollback/scale actions on an actual cluster, not just simulated ones.
 - **Slack / Jira / GitHub integrations** — notify teams and create tickets automatically where they already work.
 - **Anomaly detection model** — a learned model to catch unusual patterns that fixed rules would miss.
 - **Advanced multi-agent workflow** — separate specialist agents (investigator, postmortem writer, cost analyst) collaborating.
 - **Cost optimization agent** — an agent that watches cloud spend and proposes savings the same way it proposes incident fixes.
 - **Self-healing system** — for well-understood, low-risk incident types, allow fully automatic resolution without waiting for approval.
 - **Enterprise SSO** — support company-wide single sign-on for larger organizations.
-

18 Appendix: Five System Diagrams

Each diagram below is shown visually and as ready-to-paste Mermaid code — paste the code into GitHub, Notion, or the Mermaid Live Editor to get an editable, rendered version for your report or slides.

Diagram 1 — High-Level Architecture

In simple words: data flows in one direction, top to bottom. The customer's app uses the SDK to send data in; the platform stores and processes it; the AI investigates; a human approves; and the dashboard shows everything that happened.

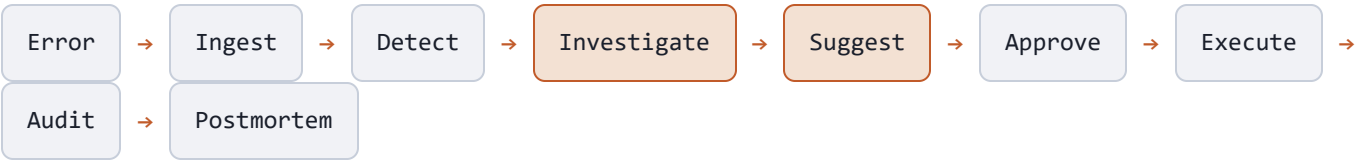


MERMAID CODE

```
flowchart TD
    SDK[Java SDK / Spring Boot Starter] --> ING[Ingestion API]
    ING --> DB[(PostgreSQL + Message Queue)]
    DB --> IE[Incident Engine]
    IE --> AI[AI Agent + RAG]
    AI --> AUTO[Automation + Human Approval]
    AUTO --> DASH[Dashboard + Audit Logs]
```

Diagram 2 — Incident Workflow

In simple words: this is the journey of one problem, from the moment it happens to the moment it's fully documented. Nine stages, always in this order.

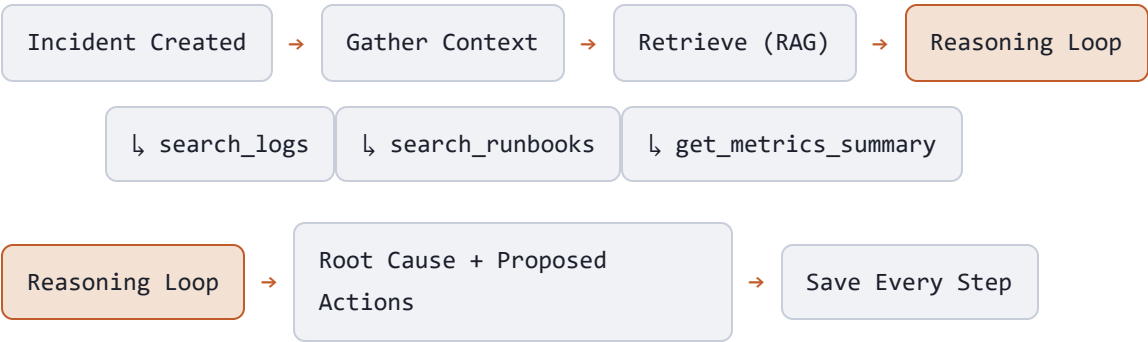


MERMAID CODE

```
flowchart LR
    A[Error Occurs] --> B[Ingest]
    B --> C[Detect]
    C --> D[Investigate]
    D --> E[Suggest Fix]
    E --> F[Human Approves]
    F --> G[Execute Automation]
    G --> H[Audit Log]
    H --> I[Postmortem]
```

Diagram 3 — AI Agent Workflow

In simple words: the AI doesn't just answer once — it works in a loop. It can repeatedly call small "tools" (search logs, search runbooks, get metrics) to gather more evidence before it finally writes its conclusion, similar to how a human investigator keeps digging until they're confident.

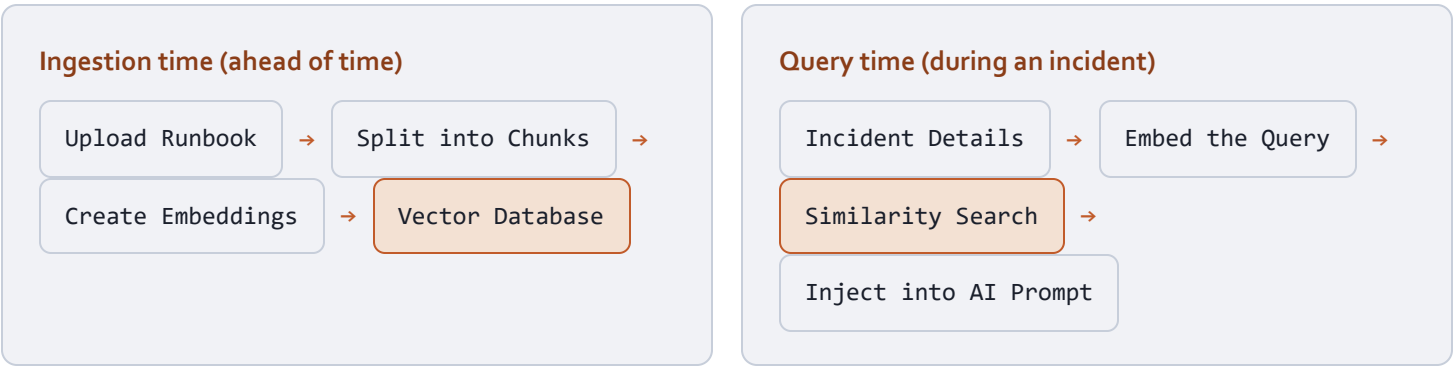


MERMAID CODE

```
flowchart TD
    T[Incident Created - Trigger] --> C[Gather Context: logs, errors, deployments]
    C --> R[Retrieve: search runbooks and past incidents]
    R --> L{LLM Reasoning Loop}
    L --> S1[search_logs]
    L --> S2[search_runbooks]
    L --> S3[get_metrics_summary]
    S1 --> L
    S2 --> L
    S3 --> L
    L --> O[Produce Output: Root Cause + Proposed Actions]
    O --> P[Persist AgentRun and AgentStep]
```

Diagram 4 — RAG Workflow

In simple words: RAG has two separate moments. First, ahead of time, documents are chopped up and turned into embeddings and stored (*ingestion time*). Later, when the AI needs an answer, it turns the question into an embedding too and searches for the closest matches (*query time*) — those matches get added to what the AI reads before it answers.



MERMAID CODE

```
graph TD
    subgraph Ingestion_Time [Ingestion Time]
        U[Upload Runbook] --> CH[Split into Chunks]
        CH --> EM[Create Embeddings]
        EM --> VDB[(Vector Database)]
    end

    subgraph Query_Time [Query Time]
        Q[Incident Details] --> QE[Embed the Query]
        QE --> SS[Similarity Search in Vector DB]
        VDB --> SS
        SS --> INJ[Inject Matching Text into AI Prompt]
    end
```

Diagram 5 — Module Dependency Diagram

In simple words: this shows which modules need which other modules to already exist. Read it bottom-to-top: the Dashboard needs almost everything else; the Audit Log module is used by nearly every other module; the AI Agent module needs both the Incident module and the Knowledge Base module to do its job.



MERMAID CODE

flowchart TD

```
Auth[Auth Module] --> OrgProj[Organization/Project Module]
OrgProj --> ApiKey[API Key Module]
OrgProj --> Ingestion[Ingestion Module]
ApiKey --> Ingestion
Ingestion --> Incident[Incident Module]
Incident --> AIAgent[AI Agent Module]
KB[Knowledge Base Module] --> AIAgent
AIAgent --> Automation[Automation Module]
Incident --> Automation
Automation --> AuditLog[Audit Log Module]
Auth --> AuditLog
Incident --> AuditLog
Ingestion --> AuditLog
Incident --> Dashboard[Dashboard Module]
AIAgent --> Dashboard
Automation --> Dashboard
AuditLog --> Dashboard
```